

```
dpp_project/
├── models/
│   ├── dpp_net.py      # CNN preprocessor model
│   └── perceptual_model.py # Pre-trained DAG-NIMA perceptual model
├── data/
│   └── vimeo_dataset.py # Vimeo-90k dataset loader
├── transform_quant.py  # Forward/inverse DCT and quantization
├── train.py            # Training script
├── inference.py        # Inference and encoding script
└── utils.py           # (Optional utilities)
```

models/dpp_net.py

The DPP network maps an input luminance frame x_t to a preprocessed output p_t . It consists of stacked convolutional layers with *varying dilation rates* openaccess.thecvf.com and PReLU activations openaccess.thecvf.com. The code below implements an example 8-layer architecture with dilations increasing then decreasing. The final output is added residually to the input (skip-connection) so that the network learns corrective residuals (this detail can be tuned).

```
# models/dpp_net.py
import torch
import torch.nn as nn

class DPPNet(nn.Module):
    """
    Deep Perceptual Preprocessing CNN for video frames.
    Operates on 1-channel (luminance) input. Uses dilated
    convolutions:contentReference[oaicite:13]{index=13}.
    """
    def __init__(self, in_channels=1, num_features=64):
        super().__init__()
        # Define convolutional layers with varying dilation rates
        self.layers = nn.ModuleList([
            nn.Sequential(nn.Conv2d(in_channels, num_features, kernel_size=3, padding=1,
dilation=1), nn.PReLU()),
            nn.Sequential(nn.Conv2d(num_features, num_features, kernel_size=3, padding=2,
dilation=2), nn.PReLU()),
```

```

        nn.Sequential(nn.Conv2d(num_features, num_features, kernel_size=3, padding=4,
dilation=4), nn.PReLU()),
        nn.Sequential(nn.Conv2d(num_features, num_features, kernel_size=3, padding=8,
dilation=8), nn.PReLU()),
        nn.Sequential(nn.Conv2d(num_features, num_features, kernel_size=3, padding=4,
dilation=4), nn.PReLU()),
        nn.Sequential(nn.Conv2d(num_features, num_features, kernel_size=3, padding=2,
dilation=2), nn.PReLU()),
        nn.Sequential(nn.Conv2d(num_features, num_features, kernel_size=3, padding=1,
dilation=1), nn.PReLU()),
        # Output layer back to 1 channel
        nn.Conv2d(num_features, in_channels, kernel_size=3, padding=1)
    ])

def forward(self, x):
    out = x
    for layer in self.layers:
        out = layer(out)
    # Residual connection (optional): add input luminance back
    p = x + out
    return p

```

This model uses **one channel** input and output (Y channel only). As in openaccess.thecvf.com, it relies solely on **dilated convolutions** to increase receptive field without downsampling, and uses PReLU activations after each conv openaccess.thecvf.com. You can adjust the number of layers or feature maps as needed.

data/vimeo_dataset.py

For training, we use the Vimeo-90k triplet dataset openaccess.thecvf.com. Below is a skeleton Dataset that loads the central frame of each triplet (or any frame) and converts it to the Y (luminance) channel. In practice, apply random cropping and multi-scale crops for **curriculum learning** openaccess.thecvf.com, but here we show basic loading:

```

# data/vimeo_dataset.py
import os
from PIL import Image
from torch.utils.data import Dataset
import torchvision.transforms.functional as TF

class Vimeo90KDataset(Dataset):

```

```

"""
Dataset for Vimeo-90K (triplet sequences). Returns luminance frames.
"""
def __init__(self, root_dir, subset='train'):
    """
    root_dir: base directory of Vimeo-90k.
    subset: 'train' or 'val'
    """
    self.root = root_dir
    self.subset = subset
    # Assume list of image paths; in practice, load from split files
    self.frame_paths = [] # populate with paths of central frames
    for line in open(os.path.join(root_dir, f"{subset}.txt")):
        seq, idx = line.strip().split()
        frame = f"img_{int(idx):04d}.png"
        self.frame_paths.append(os.path.join(root_dir, "sequences", seq, frame))

def __len__(self):
    return len(self.frame_paths)

def __getitem__(self, idx):
    path = self.frame_paths[idx]
    img = Image.open(path).convert('RGB')
    # Convert to YCbCr and take Y channel
    y, _, _ = img.convert('YCbCr').split()
    y = TF.to_tensor(y) # 1xHxW tensor, values [0,1]
    return y # Return luminance as tensor

```

Note: In practice, one should apply random **multi-scale crops** and implement curriculum learning as described in [32] (e.g. start with large easy patches, then smaller harder patches)openaccess.thecvf.com.

transform_quant.py

The virtual codec component applies a 4×4 integer DCT (as in H.264/AVCopenaccess.thecvf.com), followed by quantization and a differentiable noise model. We implement block-wise DCT transforms, quantize by dividing by a **Qstep**, and approximate rounding by adding uniform noise (width 1)openaccess.thecvf.com. An inverse transform reconstructs the frame.

```
# transform_quant.py
```

```

import torch
import torch.nn.functional as F

# Precompute 4x4 DCT transform matrix (H.264 integer DCT basis approximation)
import math

# Generate 4x4 DCT (Type-II) matrix
def dct_matrix(N=4):
    M = torch.zeros((N, N))
    for u in range(N):
        for x in range(N):
            M[u, x] = math.cos((math.pi / N) * (x + 0.5) * u)
    # Orthonormalize rows (except scale factor for u=0)
    M[0] *= 1.0 / math.sqrt(N)
    for u in range(1, N):
        M[u] *= math.sqrt(2.0 / N)
    return M

D = dct_matrix(4) # 4x4
D_inv = torch.inverse(D) # inverse DCT matrix

def block_dct(x):
    """
    Apply 4x4 DCT to non-overlapping 4x4 blocks of x (Bx1xHxW).
    Returns coefficients of same shape.
    """
    B, C, H, W = x.size()
    # Ensure divisible by 4
    assert H % 4 == 0 and W % 4 == 0
    # Unfold into blocks
    x_blocks = x.unfold(2, 4, 4).unfold(3, 4, 4) # BxCxH/4xW/4x4x4
    x_blocks = x_blocks.contiguous().view(B, C, -1, 4, 4) # BxC x numBlocks x4x4
    # Apply DCT:  $Y = D * x * D^T$ 
    # Perform for each block and channel
    Y = torch.matmul(D.to(x.device), x_blocks) # BxC x numBlocks x4x4
    Y = torch.matmul(Y, D.t().to(x.device))
    # Restore shape
    Y = Y.view(B, C, H//4, W//4, 4, 4).permute(0,1,2,4,3,5)
    Y = Y.contiguous().view(B, C, H, W)
    return Y

def block_idct(y):
    """
    Apply inverse 4x4 DCT on each 4x4 block (inverse transform).

```

```

"""
B, C, H, W = y.size()
y_blocks = y.unfold(2, 4, 4).unfold(3, 4, 4)
y_blocks = y_blocks.contiguous().view(B, C, -1, 4, 4)
# Inverse:  $X = D^T * y * D$ 
X = torch.matmul(D.t().to(y.device), y_blocks)
X = torch.matmul(X, D.to(y.device))
X = X.view(B, C, H//4, W//4, 4, 4).permute(0,1,2,4,3,5)
X = X.contiguous().view(B, C, H, W)
return X

def quantize(y, Qstep):
    """
    Quantize DCT coefficients by dividing by Qstep and rounding (during training we
    approximate rounding with uniform noise:contentReference[oaicite:21]{index=21}).
    """
    # Simulate non-differentiable rounding by adding uniform noise in [-0.5,0.5]
    noise = (torch.rand_like(y) - 0.5)
    yq = y / Qstep + noise
    return yq

def dequantize(yq, Qstep):
    """
    Inverse of quantization: multiply by Qstep.
    """
    return yq * Qstep

# Example usage in training loop:
# r = p # residual = preprocessed frame (skip prediction)
# y = block_dct(r)
# yq = quantize(y, Qstep)          # add uniform noise as rounding
proxy:contentReference[oaicite:22]{index=22}
# r_hat = block_idct(dequantize(yq, Qstep))
# p_hat = r_hat # since no prediction added (intra case)

```

This code implements the **4×4 forward and inverse DCT** transforms (approximated via real DCT) and quantization with noise. In the CVPR paper, the authors choose the H.264 integer transform matricesopenaccess.thecvf.com; here we approximate with the mathematical DCT for clarity. **Qstep** is analogous to the H.264 quantization step (mapped from QP)openaccess.thecvf.com. The **forward pass** in training uses **block_dct**, then quantize (with additive noise)openaccess.thecvf.com, then **block_idct** after dequantization to obtain the reconstructed frame $\hat{p}_t$

models/perceptual_model.py

The perceptual loss uses a **no-reference IQA model**. Following the paper, we use a *DAG-NIMA* architecture: a VGG-16 backbone (pre-trained on ImageNet) with no final fully-connected layers, followed by global average pooling and a linear layer to output a 5-category quality distribution openaccess.thecvf.com. This model is first trained on KonIQ-10k (or any NR-IQA data) and then **frozen** during DPP training openaccess.thecvf.com. We only show construction and freezing here:

```
# models/perceptual_model.py
import torch.nn as nn
from torchvision import models

class NIMA_DAG(nn.Module):
    """
    DAG variant of NIMA: VGG-16 backbone + global average pooling + linear to 5
    outputs:contentReference[oaicite:28]{index=28}.
    """
    def __init__(self, pretrained=True):
        super().__init__()
        vgg = models.vgg16(pretrained=pretrained)
        # Remove fully-connected classifier, keep features
        self.features = vgg.features
        # Global average pooling and linear layer to 5 quality classes
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(512, 5) # 5 output bins (ACR 1-5)

    def forward(self, x):
        x = self.features(x)
        x = self.pool(x)      # B x 512 x 1 x 1
        x = x.view(x.size(0), -1) # B x 512
        x = self.fc(x)        # B x 5 (logits)
        return x

# Load and freeze the perceptual model
perceptual_model = NIMA_DAG(pretrained=True)
for param in perceptual_model.parameters():
    param.requires_grad = False # freeze:contentReference[oaicite:29]{index=29}
```

During training, for an output frame $\hat{p}_t$ (converted to RGB by concatenating original U/V chroma channels), we compute the perceptual loss by passing it through this model. Let $s_i = P(\hat{p}_t)_i$ be the predicted probability of score i . The model's **predicted MOS** is $\sum_i i$

, s_i , so the perceptual loss is $-\sum_i s_i$ openaccess.thecvf.com. In code, if `logits = perceptual_model(p_rgb)`, then `probs = softmax(logits)` and `perceptual_loss = -torch.sum(probs * torch.arange(1,6,device=probs.device), dim=1).mean()`.

Loss Functions

Following Chadha *et al.*, we define a **composite loss** $L = \gamma L_P + \lambda L_R + L_F$ openaccess.thecvf.com, combining perceptual loss L_P , rate loss L_R , and fidelity loss L_F .

- Fidelity loss L_F :** A weighted sum of L1 loss and (1 – MS-SSIM) openaccess.thecvf.com:

$$L_F(x,p) = \alpha \|x - \hat{p}\|_1 + \beta (1 - \text{MS-SSIM}(x, \hat{p}))$$

$$L_F(x,p) = \alpha \|x - \hat{p}\|_1 + \beta (1 - \text{MS-SSIM}(x, \hat{p}))$$
 where α, β control luminance vs. structural preservation. (We may set $\alpha = \beta = 1$ by default, or tune as in [48].)
- Rate loss L_R :** An entropy estimate of the quantized DCT coefficients. In theory, $L_R = -E[\sum_{n,s} \log_2 p(z_{n,s})]$ where $z_{n,s}$ are normalized DCT subband coefficients openaccess.thecvf.com. For simplicity, one can approximate this by assuming a Laplace or Gaussian prior on each subband, or use a learnable density model. For example, a rough proxy is to penalize high-frequency magnitude:

Example placeholder: use L1 norm of quantized coefs as proxy for rate

- `z = (yq - yq.mean()) / (yq.std() + 1e-6)` # normalize coefficients
- `rate_loss = torch.mean(torch.abs(z))`
- In practice, one would implement $-\log_2 p(z)$ according to a prior model (as in openaccess.thecvf.com).
- Perceptual loss L_P :** Based on the frozen NIMA model. For each decoded frame $\hat{p}_{\{t\}}$ (in RGB), let s_i be the model's predicted probability of score i . We minimize the **negative MOS**:

$$L_P = -E_t \sum_{i=1}^5 i \cdot s_i(\hat{p}_t)$$

$$L_P = -E_t \sum_{i=1}^5 i \cdot s_i(\hat{p}_t)$$
 i.e. $-\sum_i i \cdot s_i$, encouraging higher perceptual score openaccess.thecvf.com.

These losses are combined as $L = L_F + \lambda L_R + \gamma L_P$ openaccess.thecvf.com. In training we alternate different λ values (e.g. in $[0.001, 0.01]$ as suggested openaccess.thecvf.com) to trade off bitrate vs. distortion. The perceptual weight γ controls emphasis on the perceptual loss. For example, one may start with $\lambda=0.005$, $\gamma=0.01$, $\alpha=\beta=1$ as a baseline.

Training Pipeline (**train.py**)

The network is trained end-to-end on Vimeo-90k with **curriculum learning** openaccess.thecvf.com. We use the **Adam optimizer** (learning rate $\sim 1e-4$) and decay when validation metrics plateau openaccess.thecvf.com. An example training loop skeleton is:

```
# train.py (simplified)

import torch

from torch.utils.data import DataLoader

from models.dpp_net import DPPNet

from models.perceptual_model import perceptual_model # NIMA model, frozen

from data.vimeo_dataset import Vimeo90KDataset

from transform_quant import block_dct, quantize, block_idct, dequantize

import torch.optim as optim

import pytorch_msssim # for MS-SSIM

# Hyperparameters

alpha, beta = 1.0, 1.0 # L1 vs. (1-MS-SSIM) weights

lam = 0.005 # rate weight
λ:contentReference[oaicite:40]{index=40}:contentReference[oaicite:41]{index=41}

gamma = 0.01 # perceptual weight γ:contentReference[oaicite:42]{index=42}

learning_rate = 1e-4
```



```

# Prepare data

dataset = Vimeo90KDataset(root_dir='path/to/vimeo90k', subset='train')

loader = DataLoader(dataset, batch_size=8, shuffle=True, num_workers=4)

model = DPPNet().cuda()

optimizer = optim.Adam(model.parameters(), lr=learning_rate)

for epoch in range(num_epochs):

    model.train()

    for x in loader:

        x = x.cuda() # Bx1xHxW

        # Forward pass through DPP network

        p = model(x) # preprocessed frame

        # Here we skip motion prediction; treat as intra-coded residual = p

        # Transform, quantize, add noise (training)

        y = block_dct(p)

        Qstep = 1.0 + torch.rand(1).item() * 9.0 # example random Qstep in [1,10]

        yq = quantize(y, Qstep)

        # Inverse transform to get reconstructed frame

        r_hat = block_idct(dequantize(yq, Qstep))

        p_hat = r_hat # since no predictor added

        # Losses

        # Fidelity: L1 + (1 - MS-SSIM)

```

```

L1 = torch.mean(torch.abs(x - p_hat))

msssim_loss = 1 - pytorch_msssim.ms_ssim(p_hat, x, data_range=1.0,
size_average=True)

L_fid = alpha * L1 + beta * msssim_loss #:contentReference[oaicite:43]{index=43}

# Rate (proxy)

z = (yq - yq.mean()) / (yq.std() + 1e-6)

L_rate = torch.mean(torch.abs(z))

# Perceptual: negative MOS from NIMA:contentReference[oaicite:44]{index=44}

# Convert p_hat to RGB by concatenating original U/V (omitted here for simplicity)

# Assume rgb_tensor is created accordingly (shape Bx3xHxW)

rgb_tensor = torch.cat([p_hat, p_hat, p_hat], dim=1) # placeholder

logits = perceptual_model(rgb_tensor)

probs = torch.softmax(logits, dim=1)

mos = torch.sum(probs * torch.arange(1,6,device=probs.device).float(), dim=1)

L_percep = -mos.mean()

# Combined loss

loss = L_fid + lam * L_rate + gamma * L_percep #:contentReference[oaicite:45]{index=45}

optimizer.zero_grad()

loss.backward()

optimizer.step()

# Adjust learning rate if needed; evaluate on val set for
scheduling:contentReference[oaicite:46]{index=46}

# (Optional: print losses and save model checkpoints)

```

In practice, one would implement the **curriculum learning schedule**: start with larger, easier crops and gradually use smaller, harder cropsopenaccess.thecvf.com. The training alternates *mostly* inter-prediction, with an intra-coded frame every 100 iterations as in the paperopenaccess.thecvf.com. We omit motion-search here, consistent with “skip inter/intra prediction” to simplify, treating each frame as an I-frame. The **Adam optimizer** with LR decay on plateau is usedopenaccess.thecvf.com. We also vary `lam` across multiple runs (e.g. 0.001–0.01) to obtain different rate-distortion pointsopenaccess.thecvf.com.

Integration and Encoding Examples (`inference.py`)

After training, the DPP model can process test frames and the outputs can be encoded with standard codecs. Below is an example inference script that:

1. Loads a trained DPP model.
2. Reads a sequence of input frames, applies the model.
3. Saves the preprocessed frames.
4. Calls FFmpeg (and VVenc) to encode these frames with H.264, AV1, and VVC.

```
# inference.py

import os

import subprocess

from PIL import Image

import torch

from models.dpp_net import DPPNet

# Load trained model

model = DPPNet().cuda()

model.load_state_dict(torch.load('dpp_model.pth'))
```

```
model.eval()
```

```
# Process test video frames (assume frames are in 'input_frames/' directory)
```

```
input_dir = 'input_frames'
```

```
output_dir = 'dpp_frames'
```

```
os.makedirs(output_dir, exist_ok=True)
```

```
for filename in sorted(os.listdir(input_dir)):
```

```
    if not filename.endswith('.png'): continue
```

```
    img = Image.open(os.path.join(input_dir, filename)).convert('YCbCr')
```

```
    y, u, v = img.split()
```

```
    y_tensor = TF.to_tensor(y).unsqueeze(0).cuda() # 1x1xHxW
```

```
    with torch.no_grad():
```

```
        p = model(y_tensor)
```

```
    p_img = TF.to_pil_image(p.squeeze(0).cpu().clamp(0,1))
```

```
    # Recombine with original U/V channels to save as color image (optional)
```

```
    p_yuv = Image.merge('YCbCr', [p_img, u, v]).convert('RGB')
```

```
    p_yuv.save(os.path.join(output_dir, filename))
```

```
# Integration examples: encode DPP output with
```

```
FFmpeg/VVenc:contentReference[oaicite:51]{index=51}:contentReference[oaicite:52]{index=52}
}
```

```
# Encode with H.264 (libx264) using CRF mode:contentReference[oaicite:53]{index=53}
```

```
subprocess.call([
```

```
'ffmpeg', '-y', '-i', f'{output_dir}/frame_%04d.png',  
'-c:v', 'libx264', '-preset', 'veryslow', '-crf', '23',  
'output_h264.mp4'  
)
```

```
# Encode with AV1 (libaom-av1) at target bitrate (kbps):contentReference[oaicite:54]{index=54}
```

```
target_kbps = 1000 # example bitrate
```

```
subprocess.call([  
    'ffmpeg', '-y', '-i', f'{output_dir}/frame_%04d.png',  
    '-c:v', 'libaom-av1', '-b:v', f'{target_kbps}k', '-cpu-used', '4',  
    'output_av1.webm'  
)
```

```
# Encode with VVC using VVEnc (YUV format):contentReference[oaicite:55]{index=55}
```

```
# First convert frames to raw YUV420
```

```
subprocess.call([  
    'ffmpeg', '-y', '-i', f'{output_dir}/frame_%04d.png',  
    '-pix_fmt', 'yuv420p', 'temp.y4m'  
)
```

```
# Run VVEnc (assuming vvencapp is installed):contentReference[oaicite:56]{index=56}
```

```
subprocess.call([  
    'vvencapp', '-i', 'temp.y4m', '-s', '1920x1080', '-c', 'yuv420',  
    '--preset', 'slow', '--qp', '27', '-ip', '128', '-o', 'output_vvc.bin'  
)
```

These commands follow the encoding **recipes** given in the supplementary material: H.264 via [libx264](https://openaccess.thecvf.com) with CRFopenaccess.thecvf.com, AV1 via [aomenc](https://openaccess.thecvf.com)/FFmpeg with target bitrateopenaccess.thecvf.com, and VVC via [vvcencappopenaccess.thecvf.com](https://openaccess.thecvf.com). The DPP-processed frames are decoded normally by the codec, and the bitstreams can be evaluated with any metric (SSIM, VMAF, etc.) to measure BD-rate savings as inopenaccess.thecvf.com.